Pay1oad 악성코드 스터디

악성코드 분석 스터디 4주차
“윈도우 실행파일의 구조(PE) 심화”

CONTENTS

01 • 윈도우 실행파일
PE

02 • PE 역사
MS의 역사

03 • PE 요소
PE의 구조

04 • None
• None

05 • Q&A

Chapter. • 1

윈도우 실행파일

윈도우 실행파일

확장자	전체 이름 (Full Name)	설명
.exe	Executable	윈도우의 가장 기본적인 프로그램 실행 파일입니다. 사용자가 더블 클릭하여 프로그램을 구동할 때 사용됩니다.
.com	Command	주로 과거 MS-DOS 환경에서 사용되던 크기가 작은 구형 명령 실행 파일입니다.
.dll	Dynamic Link Library	여러 프로그램이 공통으로 사용하는 코드와 데이터를 모아둔 동적 링크 라이브러리 파일입니다. 프로그램 실행 시 필요한 기능만 불러와 메모리를 효율적으로 사용하게 해줍니다.
.sys	System	윈도우 운영체제 구동에 필수적인 시스템 파일이거나 핵심 하드웨어 장치 드라이버입니다.
.acm	Audio Compression Manager	윈도우에서 오디오 데이터의 코덱(인코딩/디코딩)이나 압축을 처리하는 오디오 압축 드라이버 파일입니다.
.ax	DirectShow Filter	비디오 및 오디오의 재생, 변환, 압축 해제 등을 처리하기 위해 사용되는 다이렉트쇼 필터 파일입니다.
.cpl	Control Panel	윈도우 제어판 내에 있는 개별 설정 메뉴(예: 디스플레이 설정, 마우스 설정 등)를 실행하는 제어판 앱릿 파일입니다.
.drv	Driver	컴퓨터에 연결된 하드웨어 장치를 운영체제에서 인식하고 제어할 수 있게 돕는 장치 드라이버 파일입니다.
.efi	Extensible Firmware Interface	메인보드의 UEFI 펌웨어 환경에서 운영체제 부팅을 담당하는 부트 로더 및 시스템 파일입니다.
.mui	Multilingual User Interface	윈도우 화면의 메뉴, 대화 상자 등을 다양한 국가의 언어로 표시하기 위해 텍스트 리소스를 담아둔 다국어 지원 파일입니다.
.ocx	OLE Control Extension	인터넷 익스플로러나 윈도우 응용 프로그램에서 특정 기능을 추가로 수행할 때 사용되는 ActiveX 컨트롤 파일입니다.
.scr	Screensaver	윈도우 화면 보호기 파일입니다. 확장자만 다를 뿐 실제 작동 방식은 .exe 실행 파일과 같습니다.
.tsp	Telephony Service Provider	윈도우에서 모뎀이나 전화망을 통한 통신 장비를 제어할 때 사용하는 전화 통신 서비스 제공자 파일입니다.
.msstyles	Microsoft Visual Styles	윈도우의 테마, 창의 테두리 색상, 버튼 디자인 등 전반적인 UI 디자인을 지정하는 시각 스타일(테마) 파일입니다.

Chapter. • 2

PE 리뷰

Physical/Virtual Memory

RamMap - Sysinternals: www.sysinternals.com

File Empty Settings Help

Use Counts | Processes | Priority Summary | Physical Pages | Physical Ranges | File Summary | File Details

Usage	Total	Active	Standby	Modifie
Process Private	3,762,256 K	3,265,544 K	47,788 K	44
Mapped File	8,566,596 K	1,748,280 K	6,817,824 K	
Shareable	177,932 K	81,792 K	1,876 K	9
Page Table	175,888 K	173,664 K		
Paged Pool	488,428 K	475,056 K		1
Nonpaged Pool	507,644 K	507,644 K		
System PTE	505,216 K	505,088 K		
Session Private				
Metafile	224,808 K	150,336 K	74,468 K	
AWE				
Driver Locked	67,540 K	67,540 K		
Kernel Stack	81,372 K	75,576 K		
Unused	18,917,876 K	22,784 K		
Large Page				
Total	33,475,556 K	7,073,304 K	6,941,956 K	56

VMMap - Sysinternals: www.sysinternals.com

File Edit View Tools Options Help

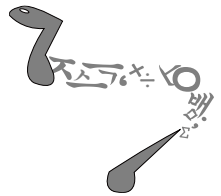
Process: chrome.exe
PID: 13692

Committed: 747,460 K
Private Bytes: 81,592 K
Working Set: 216,412 K

Type	Size	Committed	Private	Total WS	Priva
Total	2,354,087,680 K	747,460 K	81,592 K	216,412 K	7
Free	135,084,865,728 K				
Heap	14,828 K	9,380 K	9,316 K	9,016 K	
Image	412,480 K	411,528 K	8,348 K	124,684 K	
Managed Heap					
Mapped File	145,280 K	145,280 K		11,112 K	
Page Table					
Private Data	205,572,404 K	62,844 K	62,556 K	59,116 K	5
Shareable	2,147,544,656 K	117,056 K		11,676 K	
Stack	385,024 K	1,372 K	1,372 K	808 K	
Unusable	13,008 K				

Address	Type	Size	Commi...	Private	Total WS	Privat
000000007FFE0000	Private Data	4 K	4 K	4 K	4 K	
000000007FFEE000	Private Data	4 K	4 K	4 K	4 K	
0000007DBEE00000	Private Data	2,048 K	380 K	380 K	380 K	38
0000007DBF000000	Thread Stack	8,192 K	84 K	84 K	72 K	7
0000007DBF800000	Thread Stack	8,192 K	24 K	24 K	12 K	1
0000007DC0000000	Thread Stack	8,192 K	24 K	24 K	12 K	1
0000007DC0800000	Thread Stack	8,192 K	20 K	20 K	8 K	
0000007DC1000000	Thread Stack	8,192 K	20 K	20 K	8 K	

Timeline... Heap Allocations... Call Tree... Trace...



COM/PE DOS STUB

https://github.com/corkami/pics

COMMAND FILE / PE DOS STUB

ANGE ALBERTINI
http://www.corkami.com

LOADED AT CS:0100 1 0E 2 1F 3 BA 0E 01 4 B4 09 5 CD 21 6 B8 01 4C 7 CD 21

X86 (16BITS)

```
push CS
pop  DS

mov  DX, 0x10E
mov  AH, 9
int  0x21

mov  AX, 0x4C01
int  0x21
```

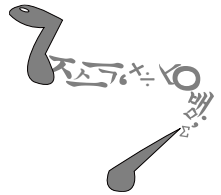
EQUIVALENT C CODE

```
// DATA segment = CODE segment

msg
print("This program ...");

return 1;
```

OFFSET 000e
ADDRESS CS:010e

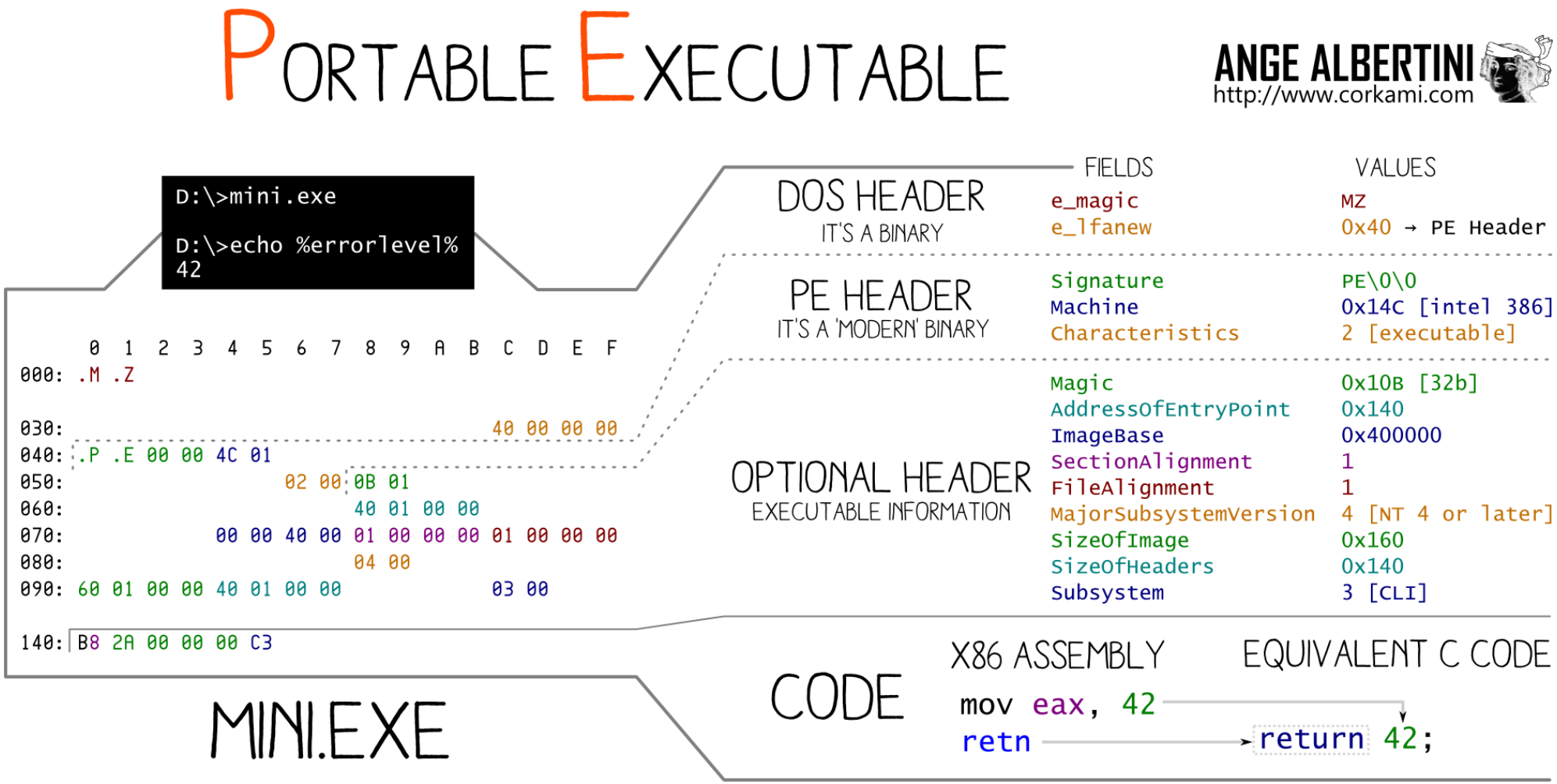
```
msg:        // ($-terminated string)
 This program cannot be run in DOS mode.\r\r\n$
```

0D 0D 0A



PE(1993)

https://github.com/corkami/pics



PE101 윈도우 실행 정보

Ange Albertini
corkami.com

PE 이해

simple

simple.exe

헤더 — 실행파일의 주요정보

DOS 헤더 — 바이너리 영역

PE 헤더 — 바이너리 영역

Optional 헤더 — 실행 정보

데이타 디렉토리 — 추가적인 구조체 정보의 주소

섹션 테이블 — 파일이 메모리에 어떤식으로 올라갈지에 대해 정의

섹션 — 실행파일의 내용

코드 영역 — 코드가 어떻게 실행되는지?

임포트 영역 — 실행시 (윈도우즈)라이브러리와 연결

데이타 영역 — 코드에서 사용되는 정보

헥사값 덤프	ASCII 덤프	항목(Fields)	값	설명
4D 5A 00 00-00 00 00 00-00 00 00 00-00 00 00 00	MZ.....	e_magic	'MZ'	고정 시그니처(MZ)
00 00 00 00-00 00 00 00-00 00 00 00-40 00 00 00	.....@...	e_lfanew	0x40	PE 헤더 주소 ❶
50 45 00 00-4C 01 03 00-00 00 00 00-00 00 00 00	PE..L.....	Signature	'PE', 0, 0	고정 시그니처(P,E,'0','0')
		Machine	0x14c [intel 386]	프로세서정보 : ARM/MIPS/Intel/...
		NumberOfSections	3	섹션의 갯수 ❷
00 00 00 00-E0 00 02 01	....a...	SizeOfOptionalHeader	0xe0	섹션 테이블의 위치와 관련있음(=Optional 헤더의 크기) ❷
		Characteristics	0x102 [32b EXE]	EXE/DLL/...
0B 01	.....	Magic	0x10b [32b]	32비트 / 64비트
00 00 00 00-00 00 00 00-00 10 00 00-00 00 00 00	.....	AddressOfEntryPoint	0x1000	실행시 코드 시작주소(지입점) ❺
00 00 00 00-00 00 40 00-00 10 00 00-00 02 00 00	.....@.....	ImageBase	0x400000	파일이 메모리에 로딩될때의 할당되는 메모리 영역의 (시작)주소
00 00 00 00-00 00 00 00-04 00 00 00-00 00 00 00	.....	SectionAlignment	0x1000	메모리 로딩시, 메모리 블럭크기(이값 단위로 메모리 할당) ❷
00 40 00 00-00 02 00 00-00 00 00 00-02 00 00 00	.@.....	FileAlignment	0x200	메로이에 로딩시, 매핑되는 파일의 블럭크기(이값 단위로 파일을 분리함) ❷
00 00 00 00-00 00 00 00-00 00 00 00-00 00 00 00	.....	MajorSubsystemVersion	4 [NT 4 or later]	요구되는 윈도우 버전
00 00 00 00-10 00 00 00	.....	SizeOfImage	0x4000	요구되는 전체 메모리크기
		SizeOfheaders	0x200	총 헤더의 크기 ❸
		Subsystem	2 [GUI]	드라이버, GUI, 커맨드 라인 ...
		NumberOfRvaAndSizes	16	데이타 디렉토리의 갯수 ❹
00 00 00 00-00 00 00 00-00 00 00 00-00 00 00 00	.....	ImportsVA	0x2000	임포트영역의 RVA* ❹
2E 74 65 78-74 00 00 00	.text...			
00 10 00 00-00 10 00 00-00 02 00 00-00 02 00 00	.....			
00 00 00 00-00 00 00 00-20 00 00 60	.....			
2E 72 64 61-74 61 00 00-00 10 00 00-00 20 00 00	.rdata.....			
00 02 00 00-00 04 00 00-00 00 00 00-00 00 00 00	.....			
00 00 00 00-40 00 00 40-2E 64 61 74-61 00 00 00	....@..@.data...			
00 10 00 00-00 30 00 00-00 02 00 00-00 06 00 00	.....0.....			
00 00 00 00-00 00 00 00-00 00 00 00-40 00 00 C0	.....@..+			

섹션 테이블(Section Table)

Name	Virtualsize (RVA*)	VirtualAddress (RVA*)	SizeOfRawData (파일에서 데이타의 크기)	PointerToRawData (파일에서 위치)	Characteristics
.text	0x1000	0x1000	0x200	0x200	CODE EXECUTE READ
.rdata	0x1000	0x2000	0x200	0x400	INITIALIZED READ
.data	0x1000	0x3000	0x200	0x600	DATA READ WRITE

각 섹션의 PointerToRawData 위치(Offset) 에서 SizeofRawData 크기만큼 읽어 들인다.
이것은 ImageBase + VirtualAddress 메모리 영역에 VirtualSize 크기만큼 로드되며, characteristics 속성을 포함하게 된다.

x86 어셈블리	C 언어 형식
push 0	
push 0x403000	
push 0x403017	
push 0	
call [0x402070]	MessageBox(0, "Hello World!","a simple PE executable", 0);
push 0	
call [0x402068]	ExitProcess(0);

임포트 구조체 / 결과

디스크립터(descriptors)

0x203c	0x204c, 0INT*		
0x2078	kernel32.dll	0,ExitProcess (Hint,Name)	
0x2068	0x204c, 0INT*		
0x2044	0x205a, 0INT*		
0x2085	user32.dll	0,MessageBoxA (Hint,Name)	
0x2070	0x205a, 0INT*		
0 0 0 0 0			

로딩후
메모리주소 0x402068 는 kernel32.dll 의 ExitProcess 를 가리키게 됩니다.
메모리주소 0x402070 는 user32.dll 의 MessageBoxA 를 가리키게 됩니다.

여기에 있는 모든주소는 RVA 입니다.*

문자열

```
a simple PE executable\0
Hello world!\0
```

이것은 파일의 전체 정보 입니다. 하지만, 많은 PE 파일은 더 많은 정보를 포함하고 있습니다. 위의 정보는 간단하고 쉽게 설명한것입니다.

version 1 KO, August 2012

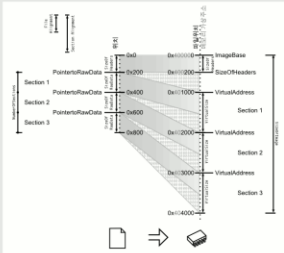
번역 <Daniel, Choi>

프로세스 로딩

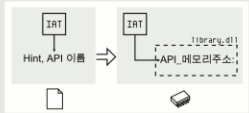
❶ 헤더
DOS 헤더 분석
PE 헤더 분석
이곳의 주소는 DOS 헤더의 e_lfanew 를 참조 합니다.)
Optional 헤더 분석
이정보는 PE 헤더 뒤에 위치합니다

❷ 섹션 테이블
섹션 테이블 분석
이곳의 위치 계산: optionalHeader 의 시작위치 + SizeOfOptionalHeader)
섹션갯수만큼(NumberOfSections) 섹션의 항목이 포함되어 있습니다.
유효한 경계(Alignment)인지 검사.:
FileAlignments, SectionAlignments

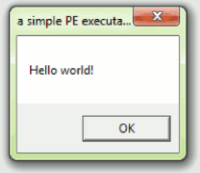
❸ 매핑
메모리에 파일이 올라갈때(mapped) 다음을 참조 합니다.
이미지베이스
헤더의크기
섹션테이블



❹ 임포트
데이타 디렉토리들 분석
데이타 디렉토리는 Optional Header 다음에 위치
그값은 RVA 와 Size 이다.
임포트 영역은 항상 두번째 항목입니다.
임포트 영역 분석
각 지시자(descriptor)는 명시된 DLL 이름.
명시된 DLL 이 메모리에 로드될시
IAT 와 INT 가 함께 분석이 됩니다.
INT(ImportNameTable)에 각각의 API 포함
이 주소는 IAT 영역에 기록되게 됩니다.



❺ 실행
엔트리포인트 에서 코드가 시작되며
코드에 존재하는 IAT 는 API 를 가리킨다.

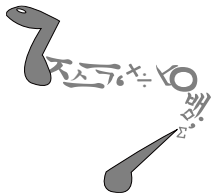


Notes

MZ 헤더 또는 DOS_헤더
'MZ'으로 시작('MZ' 는 MS-DOS 개발자 이름의(Mark Zbikowski)첫 알파벳)
PE 헤더 또는 IMAGE_FILE_HEADERS / COFF 파일 헤더
PE(Portable Executable) 로 시작함
OPTIONAL HEADER 또는 IMAGE_OPTIONAL_HEADER
Optional 헤더는 표준 PE 형식이 아니지만, 실행시 필요합니다.

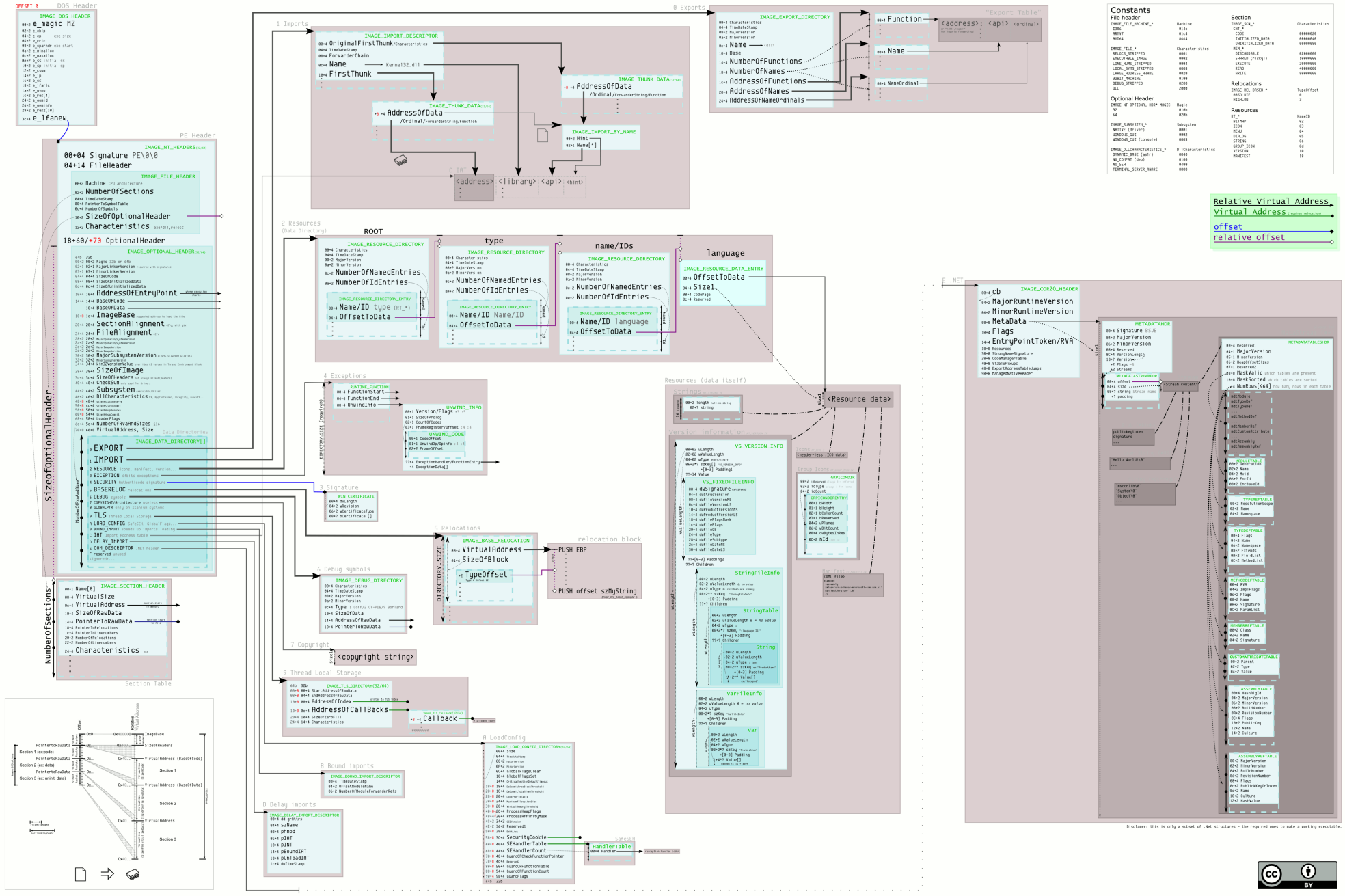
RVA Relative Virtual Address
주소는 ImageBase 와 연관됩니다.(ImageBase 는 RVA 값 0)
헤더에 존재 하는 거의 모든 주소는 코드의 RVA 입니다.
코드만에서 주소는 연관되지 않습니다.

INT Import Name Table
Null 로 끝나는 리스트는 Hint/Name 구조체를 가리킵니다.
IAT Import Address Table
Null 로 끝나는 리스트를 가리킴
INT의 복사본
로딩후, 이곳은 임포트 API 를 가리키게 됩니다.
HINT
DLL 의 익스포트 테이블에 포함되어 있는 인덱스
꼭 필요하지는 않지만, 제공될경우 검색속도가 향상됩니다.



PE102 a Windows executable format overview

Ange Albertini
2009-2014 Corkami

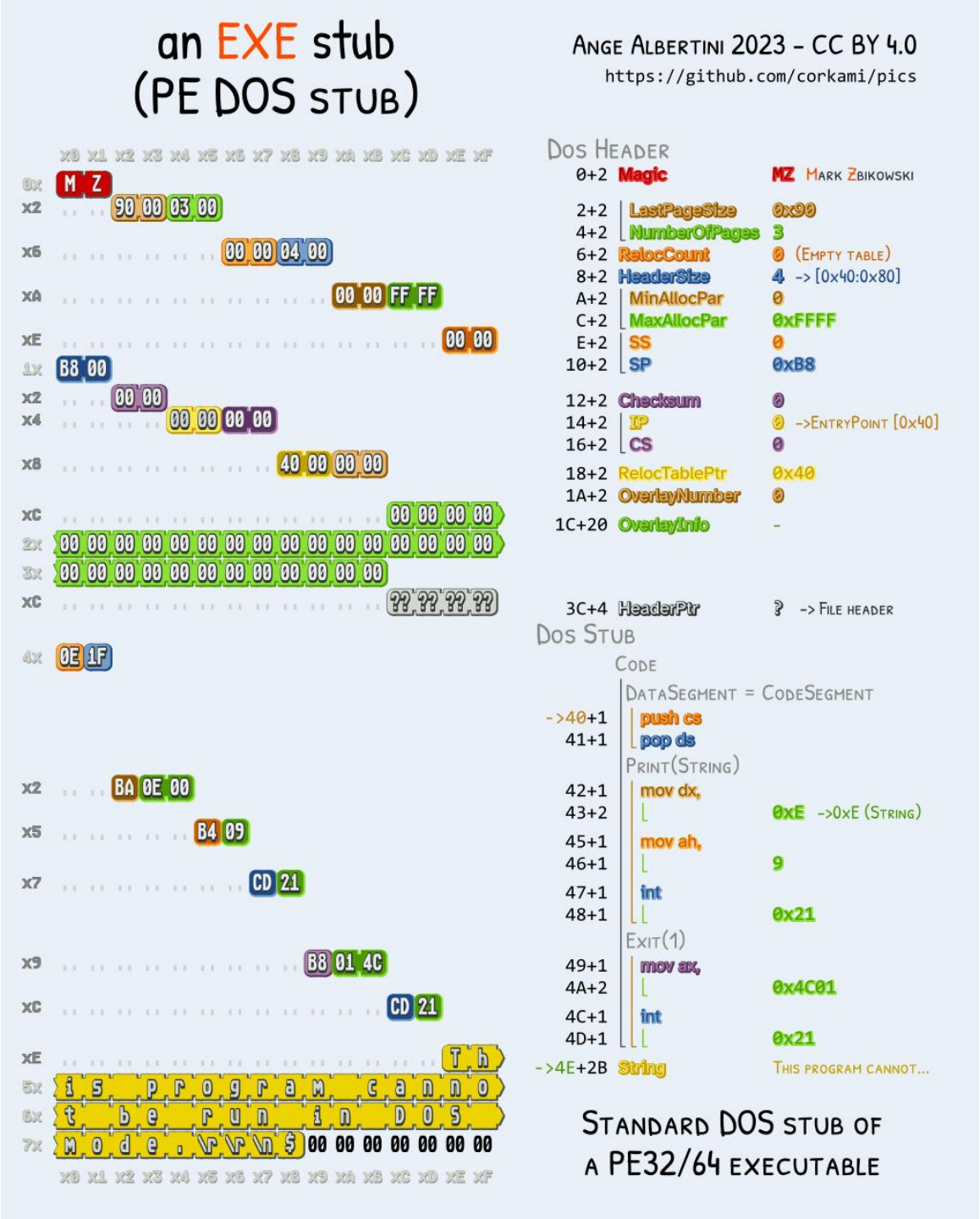


CC BY

Chapter. • 3

PE 요소

Dos Header


an EXE stub
(PE DOS STUB)
ANGE ALBERTINI 2023 - CC BY 4.0
https://github.com/corkami/pics
DOS HEADER
0+2 Magic MZ MARK ZBIKOWSKI
2+2 LastPageSize 0x90
4+2 NumberOfPages 3
6+2 RelocCount 0 (EMPTY TABLE)
8+2 HeaderSize 4 -> [0x40:0x80]
A+2 MinAllocPar 0
C+2 MaxAllocPar 0xFFFF
E+2 SS 0
10+2 SP 0xB8
12+2 Checksum 0
14+2 IP 0 ->ENTRYPOINT [0x40]
16+2 CS 0
18+2 RelocTablePtr 0x40
1A+2 OverlayNumber 0
1C+20 OverlayInfo -
3C+4 HeaderPtr ? -> FILE HEADER
DOS STUB
CODE
DATASEGMENT = CODESEGMENT
->40+1 push cs
41+1 pop ds
PRINT(STRING)
42+1 mov dx,
43+2 0xE ->0xE (STRING)
45+1 mov ah,
46+1 9
47+1 int
48+1 0x21
EXIT(1)
49+1 mov ax,
4A+2 0x4C01
4C+1 int
4D+1 0x21
->4E+2B String THIS PROGRAM CANNOT...
STANDARD DOS STUB OF
A PE32/64 EXECUTABLE

Dos Header

오프셋 (Hex)	크기 (Byte)	필드명 (표준 구조체 기준)	이미지 내 명칭	설명 및 제공된 예시 값
0x00	2	e_magic	Magic	DOS 시그니처 (Magic Number). 항상 MZ (0x5A4D) 값을 가집니다. 이는 MS-DOS 개발자인 Mark Zbikowski의 이니셜입니다.
0x02	2	e_cblp	LastPageSize	파일의 마지막 페이지의 바이트 수 (예: 0x90)
0x04	2	e_cp	NumberOfPages	파일의 총 페이지 수 (예: 3)
0x06	2	e_crlc	RelocCount	재배치(Relocation) 항목의 수 (예: 0 - Empty table)
0x08	2	e_cparhdr	HeaderSize	문단(Paragraph) 단위의 헤더 크기 (예: 4)
0x0A	2	e_minalloc	MinAllocPar	프로그램 실행에 필요한 최소 추가 메모리 (예: 0)
0x0C	2	e_maxalloc	MaxAllocPar	프로그램 실행에 필요한 최대 추가 메모리 (예: 0xFFFF)
0x0E	2	e_ss	SS	초기 SS (Stack Segment) 레지스터 값 (예: 0)
0x10	2	e_sp	SP	초기 SP (Stack Pointer) 레지스터 값 (예: 0xB8)
0x12	2	e_csum	Checksum	체크섬 (예: 0)
0x14	2	e_ip	IP	초기 IP (Instruction Pointer) 레지스터 값 (예: 0)
0x16	2	e_cs	CS	초기 CS (Code Segment) 레지스터 값 (예: 0)
0x18	2	e_lfarlc	RelocTablePtr	재배치 테이블의 파일 오프셋 (예: 0x40)
0x1A	2	e_ovno	OverlayNumber	오버레이 번호 (예: 0)
0x1C	8	e_res[4]	OverlayInfo	예약된 영역 1
0x24	2	e_oemid	-	OEM 식별자
0x26	2	e_oeminfo	-	OEM 정보
0x28	20	e_res2[10]	-	예약된 영역 2
0x3C	4	e_lfanew	HeaderPtr	실제 PE Header(NT Header)가 시작되는 오프셋 주소. (이 값을 따라가면 PE\0\0 시그니처가 나옵니다.)

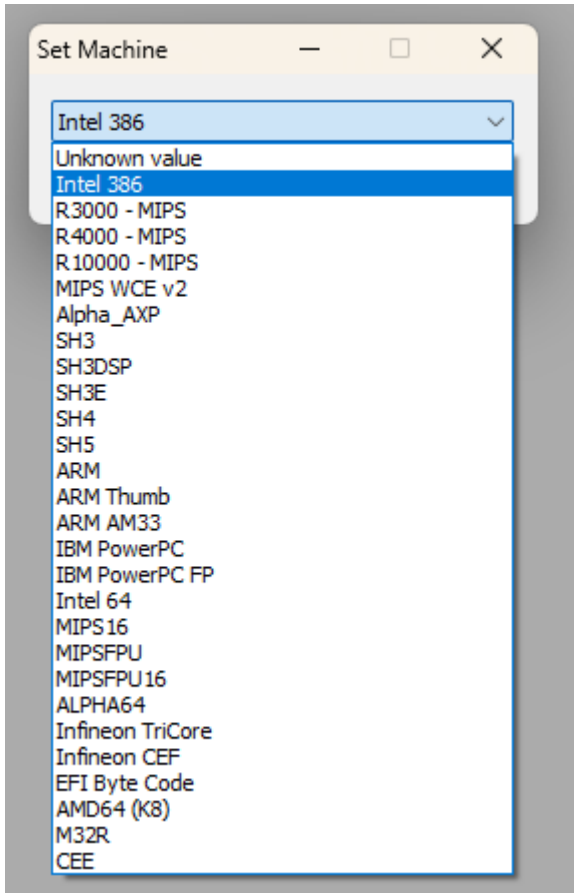
PE Header

상대 오프셋 (Hex)	크기 (Byte)	필드명 (표준 구조체 기준)	설명 및 주요 특징
0x00	4	Signature	**PE 시그니처.** 항상 PE\0\0 (0x50, 0x45, 0x00, 0x00) 값을 가집니다. 운영체제는 이 값을 통해 유효한 PE 파일인지 최종 확인합니다.
0x04	2	Machine	프로그램이 실행될 CPU 아키텍처 (예: 0x014C = Intel x86, 0x8664 = x64). 어셈블리어 명령어가 어떤 아키텍처를 기준으로 작성되었는지 OS에 알려줍니다.
0x06	2	NumberOfSections	파일 내부에 존재하는 섹션(.text, .data 등)의 총 개수입니다. 이후에 나올 섹션 헤더 배열의 크기를 결정합니다.
0x08	4	TimeDateStamp	파일이 컴파일(빌드)된 날짜와 시간입니다. (1970년 1월 1일 기준의 Unix Epoch Time 형식)
0x0C	4	PointerToSymbolTable	COFF 심볼 테이블의 오프셋 위치를 나타냅니다. 현대의 일반적인 실행 파일(EXE/DLL)에서는 보통 0으로 설정됩니다.
0x10	4	NumberOfSymbols	심볼 테이블에 있는 심볼의 개수입니다. 위와 마찬가지로 보통 0입니다.
0x14	2	SizeOfOptionalHeader	바로 뒤에 이어지는 **Optional Header의 크기**를 바이트 단위로 나타냅니다. (32비트는 보통 0xE0, 64비트는 0xF0의 값을 가집니다.)
0x16	2	Characteristics	파일의 속성을 정의하는 비트 플래그 모음입니다. (예: 0x0002 = 실행 가능 파일, 0x2000 = DLL 파일 파일, 32비트/64비트 여부 등)

PE Header 상세

https://ntcore.com/explorer-suite/

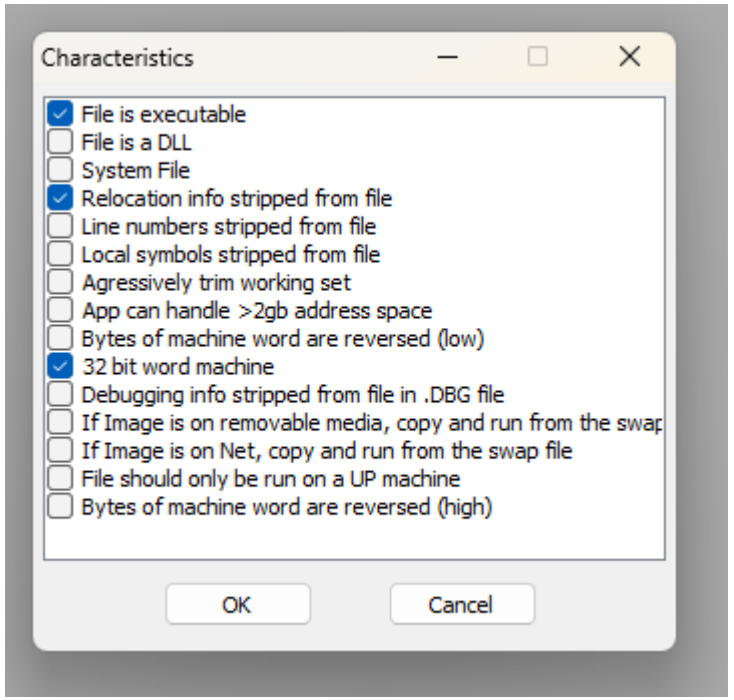
Member	Offset	Size	Value	Meaning
Machine	00000104	Word	014C	Intel 386



PE Header 상세

https://ntcore.com/explorer-suite/

Characteristics	00000116	Word	0103	Click here



PE Header 상세(공식)

https://learn.microsoft.com/en-us/windows/win32/debug/pe-format#machine-types

Constant	Value	Description
IMAGE_FILE_MACHINE_UNKNOWN	0x0	The content of this field is assumed to be applicable to any machine type
IMAGE_FILE_MACHINE_ALPHA	0x184	Alpha AXP, 32-bit address space
IMAGE_FILE_MACHINE_ALPHA64	0x284	Alpha 64, 64-bit address space
IMAGE_FILE_MACHINE_AM33	0x1d3	Matsushita AM33
IMAGE_FILE_MACHINE_AMD64	0x8664	x64
IMAGE_FILE_MACHINE_ARM	0x1c0	ARM little endian
IMAGE_FILE_MACHINE_ARM64	0xaa64	ARM64 little endian
IMAGE_FILE_MACHINE_ARM64EC	0xA641	ABI that enables interoperability between native ARM64 and emulated x64 code.
IMAGE_FILE_MACHINE_ARM64X	0xA64E	Binary format that allows both native ARM64 and ARM64EC code to coexist in the same file.
IMAGE_FILE_MACHINE_ARMNT	0x1c4	ARM Thumb-2 little endian
IMAGE_FILE_MACHINE_AXP64	0x284	AXP 64 (Same as Alpha 64)
IMAGE_FILE_MACHINE_EBC	0xebc	EFI byte code
IMAGE_FILE_MACHINE_I386	0x14c	Intel 386 or later processors and compatible processors
IMAGE_FILE_MACHINE_IA64	0x200	Intel Itanium processor family
IMAGE_FILE_MACHINE_LOONGARCH32	0x6232	LoongArch 32-bit processor family
IMAGE_FILE_MACHINE_LOONGARCH64	0x6264	LoongArch 64-bit processor family
IMAGE_FILE_MACHINE_M32R	0x9041	Mitsubishi M32R little endian

PE Header 상세(공식)

https://learn.microsoft.com/en-us/windows/win32/debug/pe-format#characteristics

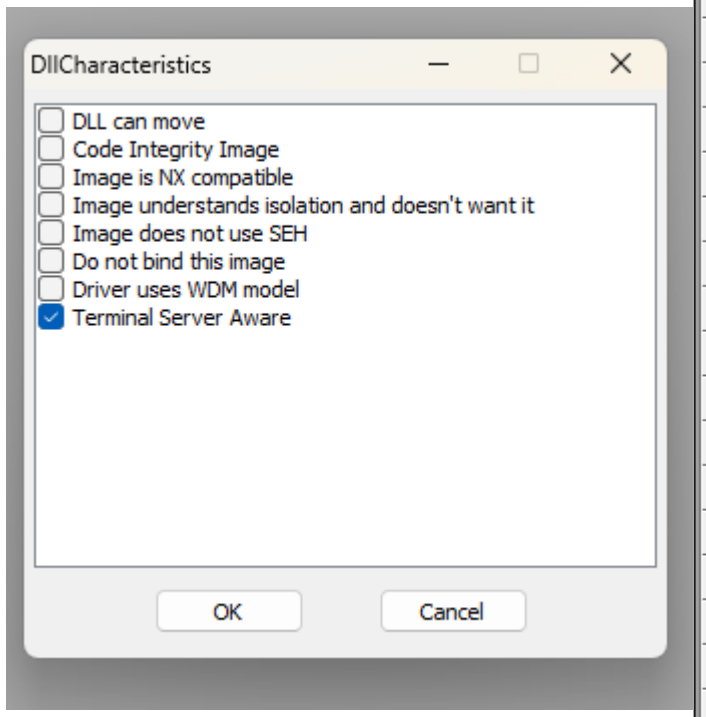
Flag	Value	Description
IMAGE_FILE_RELOCS_STRIPPED	0x0001	Image only, Windows CE, and Microsoft Windows NT and later. This indicates that the file does not contain base relocations and must therefore be loaded at its preferred base address. If the base address is not available, the loader reports an error. The default behavior of the linker is to strip base relocations from executable (EXE) files.
IMAGE_FILE_EXECUTABLE_IMAGE	0x0002	Image only. This indicates that the image file is valid and can be run. If this flag is not set, it indicates a linker error.
IMAGE_FILE_LINE_NUMS_STRIPPED	0x0004	COFF line numbers have been removed. This flag is deprecated and should be zero.
IMAGE_FILE_LOCAL_SYMS_STRIPPED	0x0008	COFF symbol table entries for local symbols have been removed. This flag is deprecated and should be zero.
IMAGE_FILE_AGGRESSIVE_WS_TRIM	0x0010	Obsolete. Aggressively trim working set. This flag is deprecated for Windows 2000 and later and must be zero.
IMAGE_FILE_LARGE_ADDRESS_AWARE	0x0020	Application can handle > 2-GB addresses.
	0x0040	This flag is reserved for future use.
IMAGE_FILE_BYTES_REVERSED_LO	0x0080	Little endian: the least significant bit (LSB) precedes the most significant bit (MSB) in memory. This flag is deprecated and should be zero.
IMAGE_FILE_32BIT_MACHINE	0x0100	Machine is based on a 32-bit-word architecture.
IMAGE_FILE_DEBUG_STRIPPED	0x0200	Debugging information is removed from the image file.
IMAGE_FILE_REMOVABLE_RUN_FROM_SWAP	0x0400	If the image is on removable media, fully load it and copy it to the swap file.
IMAGE_FILE_NET_RUN_FROM_SWAP	0x0800	If the image is on network media, fully load it and copy it to the swap file.
IMAGE_FILE_SYSTEM	0x1000	The image file is a system file, not a user program.
IMAGE_FILE_DLL	0x2000	The image file is a dynamic-link library (DLL). Such files are considered executable files for almost all purposes, although they cannot be directly run.
IMAGE_FILE_UP_SYSTEM_ONLY	0x4000	The file should be run only on a uniprocessor machine.

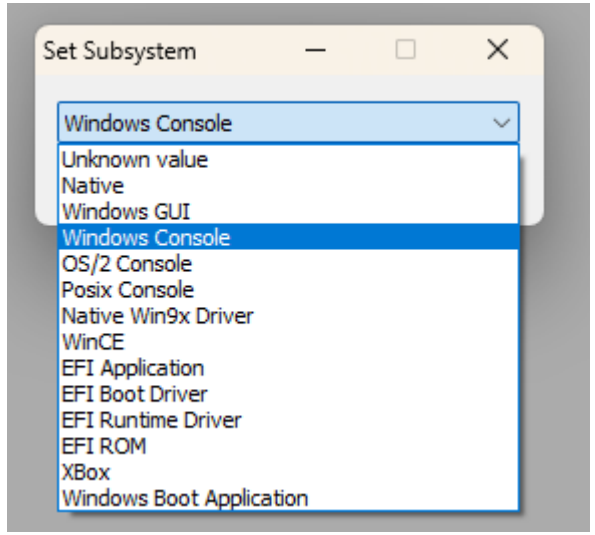
Optional Header

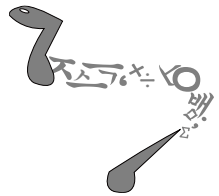
상대 오프셋 (Hex)	크기 (Byte)	필드명 (표준 구조체 기준)	설명 및 주요 특징
0x00	2	Magic	32비트 파일인지 64비트 파일인지 구분합니다. (32비트: 0x010B, 64비트: 0x020B)
0x02	1	MajorLinkerVersion	링커의 주 버전
0x03	1	MinorLinkerVersion	링커의 부 버전
0x04	4	SizeOfCode	코드(.text) 섹션의 총 크기
0x08	4	SizeOfInitializedData	초기화된 데이터(.data) 섹션의 크기
0x0C	4	SizeOfUninitializedData	초기화되지 않은 데이터(.bss) 섹션의 크기
0x10	4	AddressOfEntryPoint	**프로그램이 메모리에서 처음 실행될 코드의 주소(RVA).** 악성코드 분석이나 리버싱 시 가장 먼저 확인하는 진입점입니다.
0x14	4	BaseOfCode	코드 섹션이 시작되는 RVA 주소
0x18	4	BaseOfData	데이터 섹션이 시작되는 RVA 주소 (64비트 PE 파일에서는 이 필드가 존재하지 않습니다.)
0x1C	4	ImageBase	**PE 파일이 메모리에 로드될 때 선호하는 시작 주소입니다.** (EXE는 보통 0x400000, DLL은 0x10000000을 가집니다.)
0x20	4	SectionAlignment	**메모리(Memory)** 상에서 섹션들이 정렬되는 기본 단위(보통 0x1000 바이트 = 4KB)입니다.
0x24	4	FileAlignment	**파일(디스크)** 상에서 섹션들이 정렬되는 기본 단위(보통 0x200 바이트 = 512바이트)입니다.
0x28	8	... Version ...	OS 및 이미지, 서브시스템의 메이저/마이너 버전 정보가 포함된 영역 (총 8바이트)
0x34	4	Win32VersionValue	예약된 필드. 보통 0 값을 가집니다.
0x38	4	SizeOfImage	PE 파일이 메모리에 로딩되었을 때 차지하는 **전체 메모리 크기**입니다. (SectionAlignment의 배수여야 함)
0x3C	4	SizeOfHeaders	DOS Header, PE Header, Section Header들을 모두 합친 **전체 헤더의 크기**입니다. (FileAlignment의 배수여야 함)
0x40	4	CheckSum	파일의 무결성을 검증하기 위한 체크섬 값 (주로 드라이버 파일이나 시스템 DLL에서 중요하게 쓰입니다.)
0x44	2	Subsystem	프로그램의 동작 환경을 지정합니다. (1 = 드라이버, 2 = GUI(윈도우 프로그램), 3 = CUI(콘솔 프로그램)
0x46	2	DllCharacteristics	DLL 동작 및 보안 관련 속성 (예: ASLR, DEP 적용 여부 등)
0x48	16	SizeOf...	스택(Stack)과 힙(Heap)의 예약(Reserve) 및 할당(Commit) 크기를 지정하는 영역 (총 16바이트)
0x58	4	LoaderFlags	디버깅 지원을 위한 플래그 (보통 0)
0x5C	4	NumberOfRvaAndSizes	바로 뒤에 이어지는 **Data Directory 배열의 개수**입니다. 일반적으로 16(0x10)개의 디렉터리를 가집니다.
0x60	128	DataDirectory[16]	Export/Import 테이블, 리소스, TLS, Relocation 등 PE 파일 내의 중요 데이터 구조체 위치(RVA)와 크기를 담고 있는 16개의 배열 (각 8바이트)

Optional Header

Member	Offset	Size	Value	Meaning
Magic	00000118	Word	010B	PE32
MajorLinkerVersion	0000011A	Byte	0E	
MinorLinkerVersion	0000011B	Byte	32	
SizeOfCode	0000011C	Dword	00000C00	
SizeOfInitializedData	00000120	Dword	00001000	
SizeOfUninitializedData	00000124	Dword	00000000	
AddressOfEntryPoint	00000128	Dword	00001271	.text
BaseOfCode	0000012C	Dword	00001000	
BaseOfData	00000130	Dword	00002000	
ImageBase	00000134	Dword	00400000	
SectionAlignment	00000138	Dword	00001000	
FileAlignment	0000013C	Dword	00000200	
MajorOperatingSystemVer...	00000140	Word	0006	
MinorOperatingSystemVer...	00000142	Word	0000	
MajorImageVersion	00000144	Word	0000	
MinorImageVersion	00000146	Word	0000	
MajorSubsystemVersion	00000148	Word	0006	
MinorSubsystemVersion	0000014A	Word	0000	
Win32VersionValue	0000014C	Dword	00000000	
SizeOfImage	00000150	Dword	00005000	
SizeOfHeaders	00000154	Dword	00000400	
CheckSum	00000158	Dword	00000000	
Subsystem	0000015C	Word	0003	Windows Console
DllCharacteristics	0000015E	Word	8000	Click here
SizeOfStackReserve	00000160	Dword	00100000	
SizeOfStackCommit	00000164	Dword	00001000	
SizeOfHeapReserve	00000168	Dword	00100000	
SizeOfHeapCommit	0000016C	Dword	00001000	
LoaderFlags	00000170	Dword	00000000	
NumberOfRvaAndSizes	00000174	Dword	00000010	

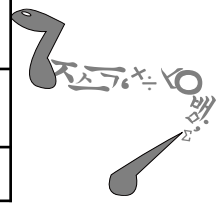






Data Directory

인덱스	오프셋 (Hex)	디렉터리 명칭	주요 역할 및 설명
0	0x60	EXPORT	프로그램이 외부(다른 프로그램)로 제공(Export)하는 함수들의 정보입니다. (주로 DLL 파일에서 중요하게 사용됨)
1	0x68	IMPORT	프로그램이 실행되기 위해 외부 DLL에서 가져와서 사용하는 함수(API)들의 정보입니다. (거의 모든 실행 파일에 필수적임)
2	0x70	RESOURCE	아이콘, 커서, 문자열, 다이얼로그 창 등 프로그램에 포함된 리소스(Resource) 데이터의 위치를 가리킵니다.
3	0x78	EXCEPTION	예외 처리(Exception Handling)와 관련된 정보입니다. (주로 64비트 환경에서 함수 테이블 참조용으로 사용됨)
4	0x80	SECURITY	파일의 디지털 서명(인증서) 정보입니다. **(유일하게 RVA가 아닌 파일의 실제 물리적 오프셋(File Offset)을 가리킵니다.)**
5	0x88	BASERELOC	메모리 로딩 시 지정된 ImageBase에 로드되지 못할 경우, 하드코딩된 주소들을 재배치(Relocation)하기 위한 정보입니다.
6	0x90	DEBUG	디버깅(Debugging) 관련 정보의 위치와 크기입니다.
7	0x98	ARCHITECTURE	특정 아키텍처에 종속적인 데이터입니다. (현재는 거의 사용되지 않으며 보통 0으로 채워짐)
8	0xA0	GLOBALPTR	Global Pointer (GP) 레지스터로 사용할 RVA 값입니다. (특정 아키텍처용, 보통 0)
9	0xA8	TLS	스레드 로컬 스토리지(Thread Local Storage) 정보입니다. 메인 스레드 실행 전 먼저 실행되는 코드를 설정할 수 있어 리버싱/안티 디버깅 기법으로 자주 분석됩니다.
10	0xB0	LOAD_CONFIG	SEH(Structured Exception Handling)나 CFG(Control Flow Guard) 등 고급 보안 및 로딩 설정 정보를 담고 있습니다.
11	0xB8	BOUND_IMPORT	로딩 속도 향상을 위해 미리 계산된 Import 주소 배열 정보입니다.
12	0xC0	IAT	**Import Address Table.** 외부에서 가져온 함수들의 실제 메모리 주소가 운영체제(로더)에 의해 덮어씌워지는 핵심 테이블입니다. (인덱스 1의 IMPORT와 밀접하게 연관됨)
13	0xC8	DELAY_IMPORT	프로그램 시작 시가 아닌, 해당 함수가 실제로 호출될 때 DLL을 로드하도록 지연시키는 정보입니다.
14	0xD0	COM_DESCRIPTOR	C# 등으로 만들어진 .NET (CLR) 프로그램의 메타데이터 정보가 담긴 곳입니다. 일반 C/C++ 프로그램에서는 사용되지 않습니다.
15	0xD8	Reserved	향후 사용을 위해 예약된 빈 공간입니다. (항상 0)



Data Directory

Member	Offset	Size	Value	Section
Export Directory RVA	00000178	Dword	00000000	
Export Directory Size	0000017C	Dword	00000000	
Import Directory RVA	00000180	Dword	000025A4	.rdata
Import Directory Size	00000184	Dword	000000A0	
Resource Directory RVA	00000188	Dword	00004000	.rsrc
Resource Directory Size	0000018C	Dword	000001E0	
Exception Directory RVA	00000190	Dword	00000000	
Exception Directory Size	00000194	Dword	00000000	
Security Directory RVA	00000198	Dword	00000000	
Security Directory Size	0000019C	Dword	00000000	
Relocation Directory RVA	000001A0	Dword	00000000	
Relocation Directory Size	000001A4	Dword	00000000	
Debug Directory RVA	000001A8	Dword	000021A0	.rdata
Debug Directory Size	000001AC	Dword	00000070	
Architecture Directory RVA	000001B0	Dword	00000000	
Architecture Directory Size	000001B4	Dword	00000000	
Reserved	000001B8	Dword	00000000	
Reserved	000001BC	Dword	00000000	
TLS Directory RVA	000001C0	Dword	00000000	
TLS Directory Size	000001C4	Dword	00000000	
Configuration Directory RVA	000001C8	Dword	000020E0	.rdata
Configuration Directory Size	000001CC	Dword	00000040	
Bound Import Directory RVA	000001D0	Dword	00000000	
Bound Import Directory Size	000001D4	Dword	00000000	
Import Address Table Directory...	000001D8	Dword	00002000	.rdata
Import Address Table Directory...	000001DC	Dword	000000A8	
Delay Import Directory RVA	000001E0	Dword	00000000	
Delay Import Directory Size	000001E4	Dword	00000000	
.NET MetaData Directory RVA	000001E8	Dword	00000000	
.NET MetaData Directory Size	000001EC	Dword	00000000	

Section Header

Name	Virtual Size	Virtual Address	Raw Size	Raw Address	Reloc Address	Linenumbers	Relocations N...	Linenumbers ...	Characteristics
Byte[8]	Dword	Dword	Dword	Dword	Dword	Dword	Word	Word	Dword
.text	00000BB3	00001000	00000C00	00000400	00000000	00000000	0000	0000	60000020
.rdata	00000A84	00002000	00000C00	00001000	00000000	00000000	0000	0000	40000040
.data	000000C4	00003000	00000200	00001C00	00000000	00000000	0000	0000	C0000040
.rsrc	000001E0	00004000	00000200	00001E00	00000000	00000000	0000	0000	40000040

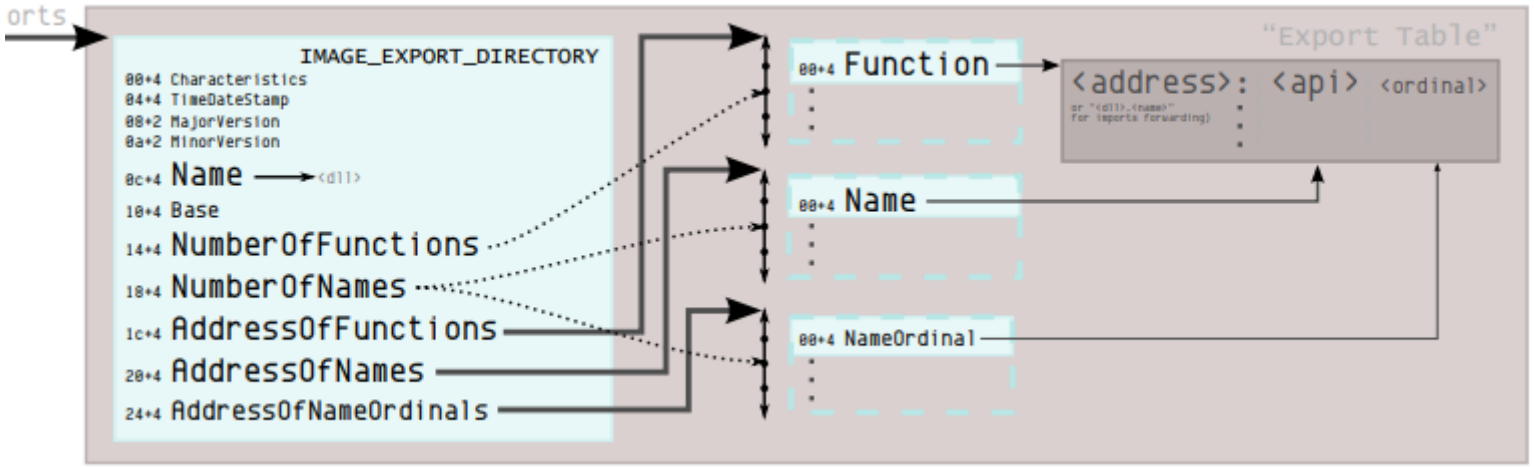
Section Header

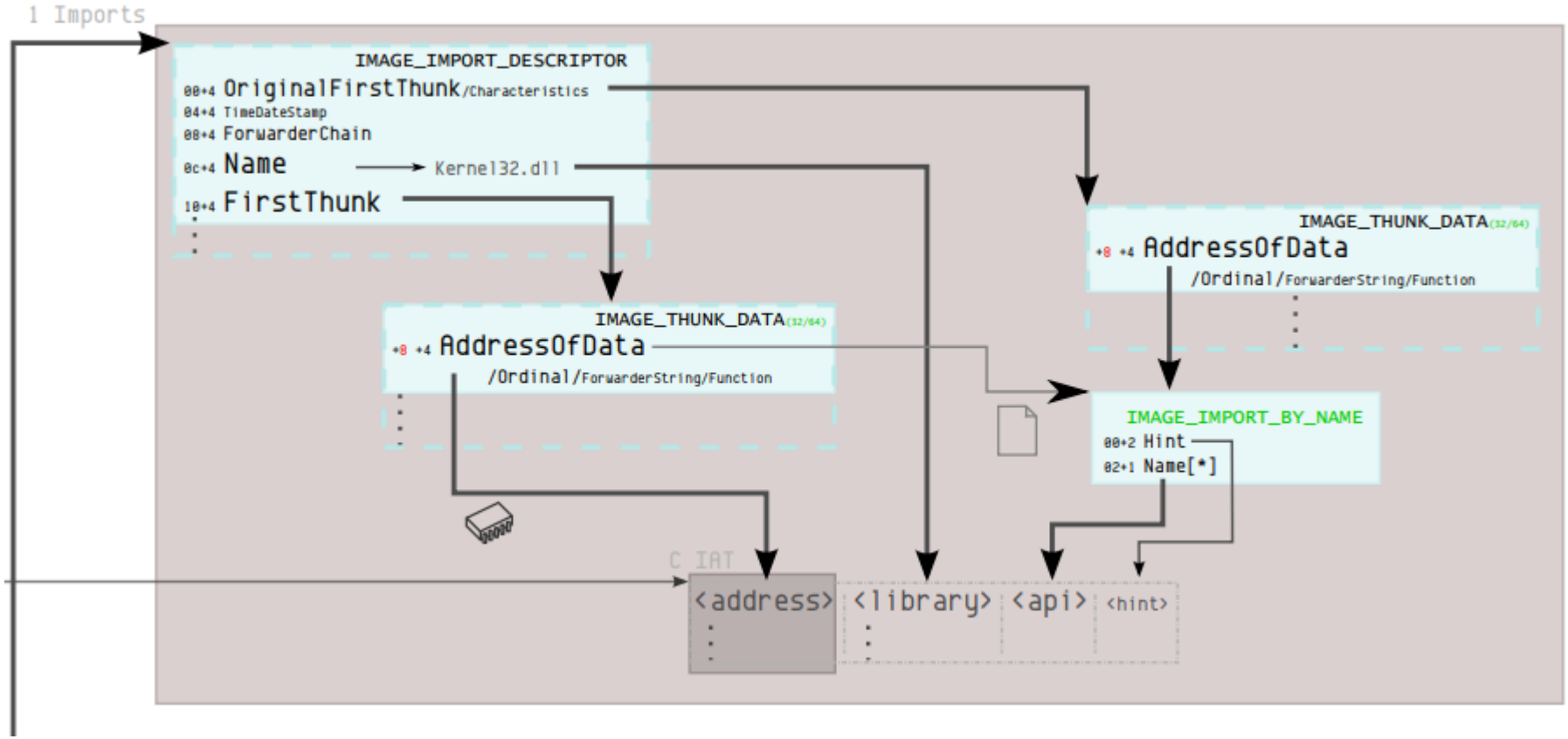
상대 오프셋 (Hex)	크기 (Byte)	필드명 (표준 구조체 기준)	설명 및 주요 특징
0x00	8	Name	섹션의 이름입니다. (예: .text, .data, .rsrc 등) 최대 8바이트의 ASCII 문자로 구성되며, 8바이트를 꽉 채울 경우 NULL 종료 문자가 생략될 수 있습니다.
0x08	4	VirtualSize	프로그램이 메모리에 로드되었을 때, 이 섹션이 차지하는 **실제 데이터의 크기**입니다. (메모리 정렬인 SectionAlignment가 적용되기 전의 순수 크기)
0x0C	4	VirtualAddress	프로그램이 메모리에 로드되었을 때, 이 섹션이 시작되는 가상 주소(RVA)입니다. (항상 SectionAlignment의 배수여야 함)
0x10	4	SizeOfRawData	파일(디스크) 상에서 이 섹션이 차지하는 **크기**입니다. (항상 FileAlignment의 배수로 패딩이 포함된 크기)
0x14	4	PointerToRawData	파일(디스크) 상에서 이 섹션의 실제 데이터가 시작되는 오프셋(File Offset)입니다. (이를 통해 디스크의 데이터를 메모리로 복사해 옵니다.)
0x18	4	PointerToRelocations	재배치 정보를 가리키는 파일 오프셋입니다. (obj 파일에서 주로 쓰이며, 일반적인 EXE에서는 0입니다.)
0x1C	4	PointerToLinenumbers	라인 번호 정보를 가리키는 파일 오프셋입니다. (디버깅용이며, 보통 0입니다.)
0x20	2	NumberOfRelocations	재배치 정보의 개수입니다. (보통 0)
0x22	2	NumberOfLinenumbers	라인 번호 정보의 개수입니다. (보통 0)
0x24	4	Characteristics	이 섹션의 **속성(권한) 플래그**입니다. (예: 코드 포함 여부, 읽기/쓰기/실행 권한 등)

IAT(Import Address Table)

Module Name	Imports	OFTs	TimeDateStamp	ForwarderChain	Name RVA	FTs (IAT)
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
VCRUNTIME140.dll	3	00002668	00000000	00000000	0000273A	00002024
api-ms-win-crt-run...	19	00002690	00000000	00000000	0000290E	0000204C
api-ms-win-crt-ma...	1	00002688	00000000	00000000	00002930	00002044
api-ms-win-crt-stdi...	2	000026E0	00000000	00000000	00002950	0000209C
api-ms-win-crt-loc...	1	00002680	00000000	00000000	00002970	0000203C
api-ms-win-crt-hea...	1	00002678	00000000	00000000	00002992	00002034
KERNEL32.dll	8	00002644	00000000	00000000	00002A76	00002000

OFTs	FTs (IAT)	Hint	Name
Dword	Dword	Word	szAnsi
00002702	00002702	001D	__current_exception_context
000026EC	000026EC	001C	__current_exception
00002720	00002720	0035	_except_handler4_common


orts
IMAGE_EXPORT_DIRECTORY
00+4 Characteristics
04+4 TimeDateStamp
08+2 MajorVersion
0a+2 MinorVersion
0c+4 Name
<dll>
10+4 Base
14+4 NumberOfFunctions
18+4 NumberOfNames
1c+4 AddressOfFunctions
20+4 AddressOfNames
24+4 AddressOfNameOrdinals
"Export Table"
00+4 Function
<address>: <api> <ordinal>
00+4 Name
00+4 NameOrdinal

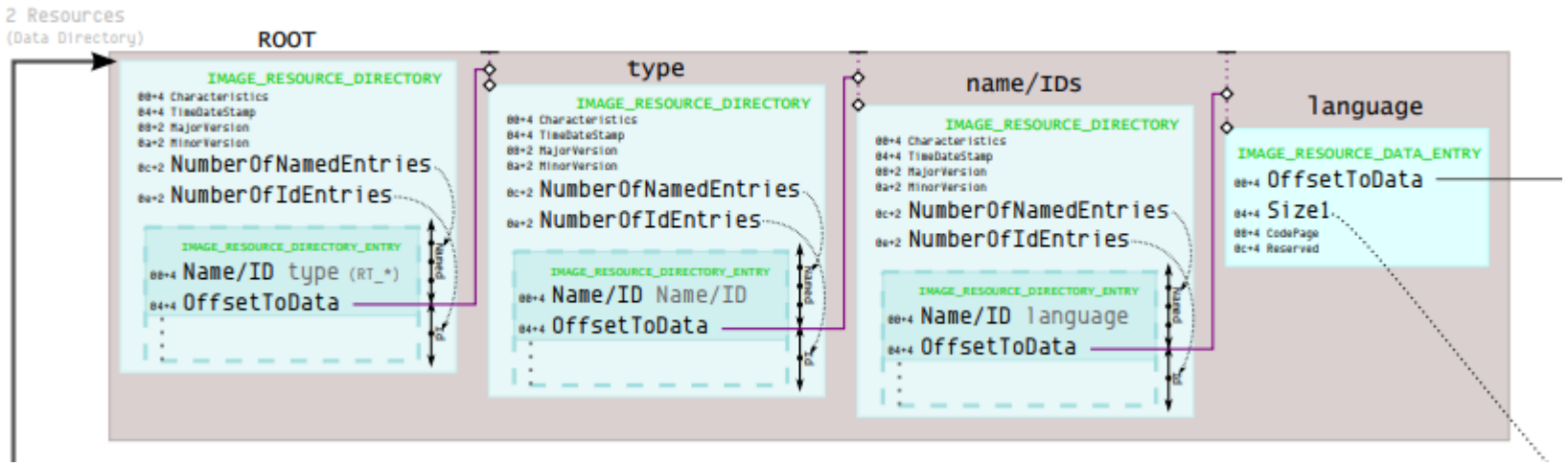

1 Imports
IMAGE_IMPORT_DESCRIPTOR
00+4 OriginalFirstThunk/Characteristics
04+4 TimeDateStamp
08+4 ForwarderChain
0c+4 Name
Kernel32.dll
10+4 FirstThunk
IMAGE_THUNK_DATA(32/64)
+8 +4 AddressOfData
/Ordinal/ForwarderString/Function
IMAGE_THUNK_DATA(32/64)
+8 +4 AddressOfData
/Ordinal/ForwarderString/Function
IMAGE_IMPORT_BY_NAME
00+2 Hint
02+1 Name[*]
C IAT
<address>
<library>
<api>
<hint>

Resource

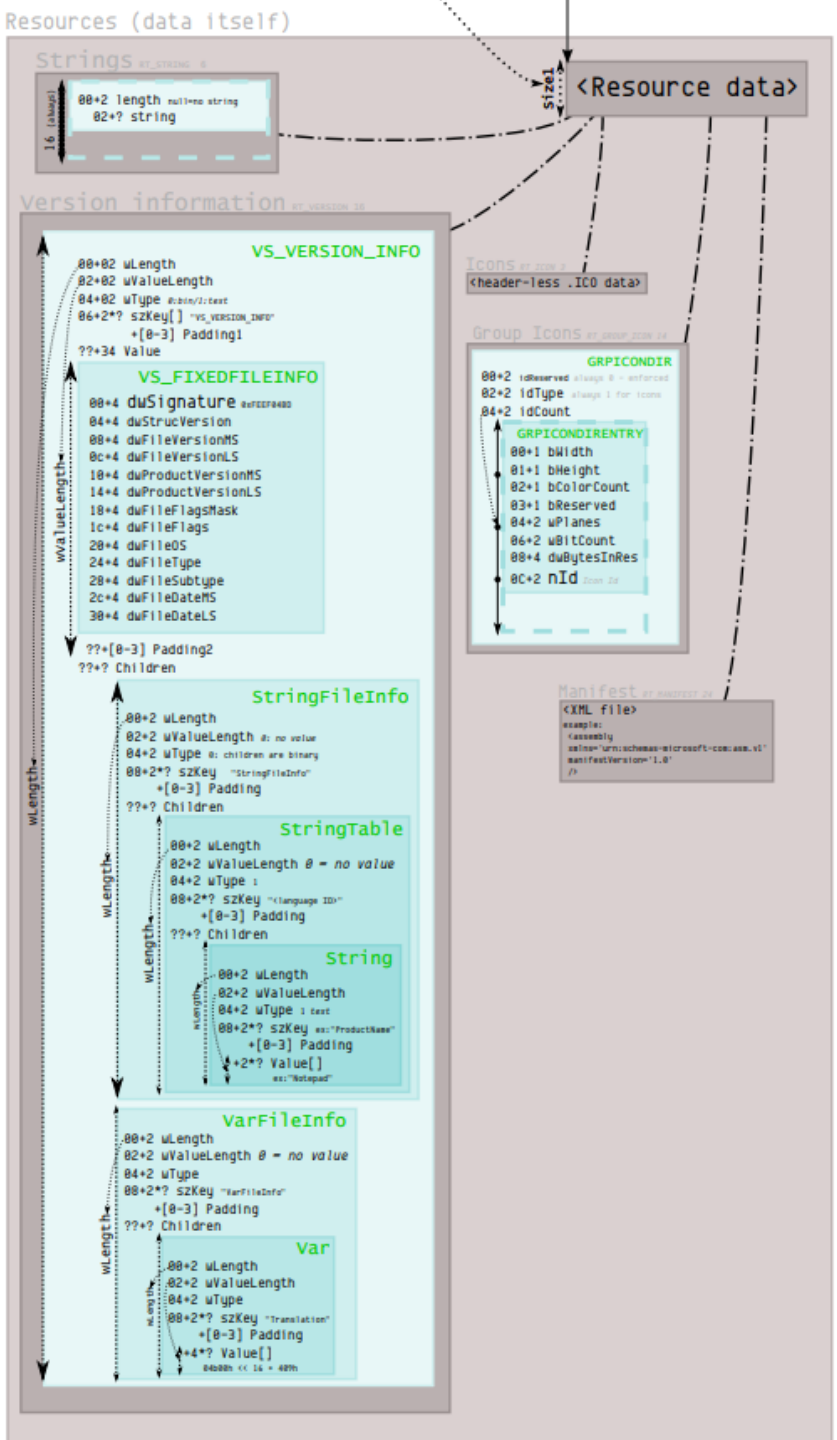
- Resource Directory
  - Resource Directory Entry 1, ID: 24, AKA: Configuration Files
    - Resource Directory
      - Resource Directory Entry 1, ID: 1
        - Resource Directory
          - Resource Directory Entry 1, ID: 1033
            - Resource Data Entry

Member	Offset	Size	Value
OffsetToData	00001E48	Dword	00004060
Size	00001E4C	Dword	0000017D
CodePage	00001E50	Dword	00000000
Reserved	00001E54	Dword	00000000

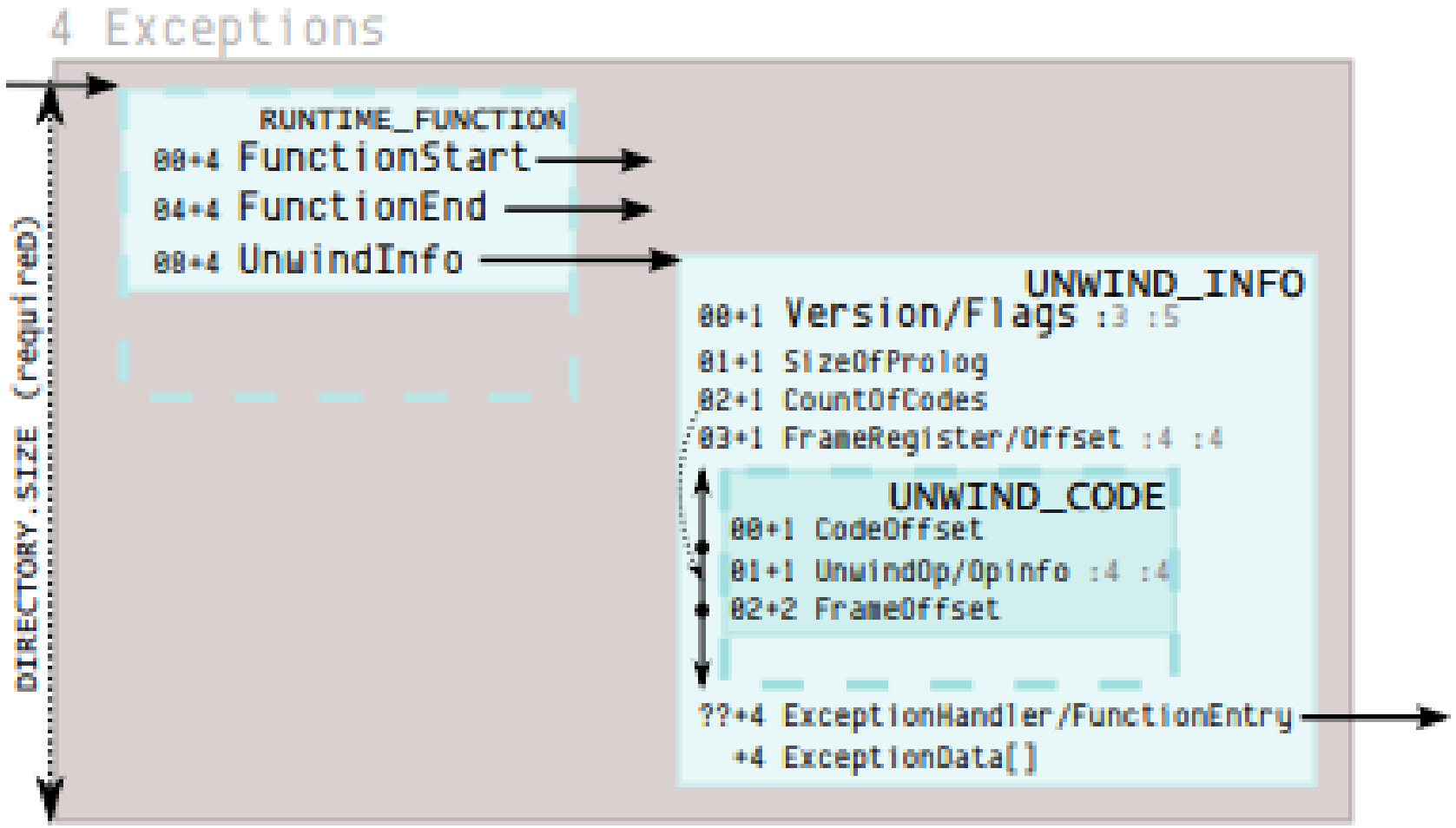
Resource


2 Resources
(Data Directory)
ROOT
IMAGE_RESOURCE_DIRECTORY
00+4 Characteristics
04+4 TimeDateStamp
08+2 MajorVersion
0a+2 MinorVersion
0c+2 NumberOfNamedEntries
0e+2 NumberOfIdEntries
IMAGE_RESOURCE_DIRECTORY_ENTRY
00+4 Name/ID type (RT_*)
04+4 OffsetToData
type
IMAGE_RESOURCE_DIRECTORY
00+4 Characteristics
04+4 TimeDateStamp
08+2 MajorVersion
0a+2 MinorVersion
0c+2 NumberOfNamedEntries
0e+2 NumberOfIdEntries
IMAGE_RESOURCE_DIRECTORY_ENTRY
00+4 Name/ID Name/ID
04+4 OffsetToData
name/IDs
IMAGE_RESOURCE_DIRECTORY
00+4 Characteristics
04+4 TimeDateStamp
08+2 MajorVersion
0a+2 MinorVersion
0c+2 NumberOfNamedEntries
0e+2 NumberOfIdEntries
IMAGE_RESOURCE_DIRECTORY_ENTRY
00+4 Name/ID language
04+4 OffsetToData
language
IMAGE_RESOURCE_DATA_ENTRY
00+4 OffsetToData
04+4 Size1
08+4 CodePage
0c+4 Reserved

Resource

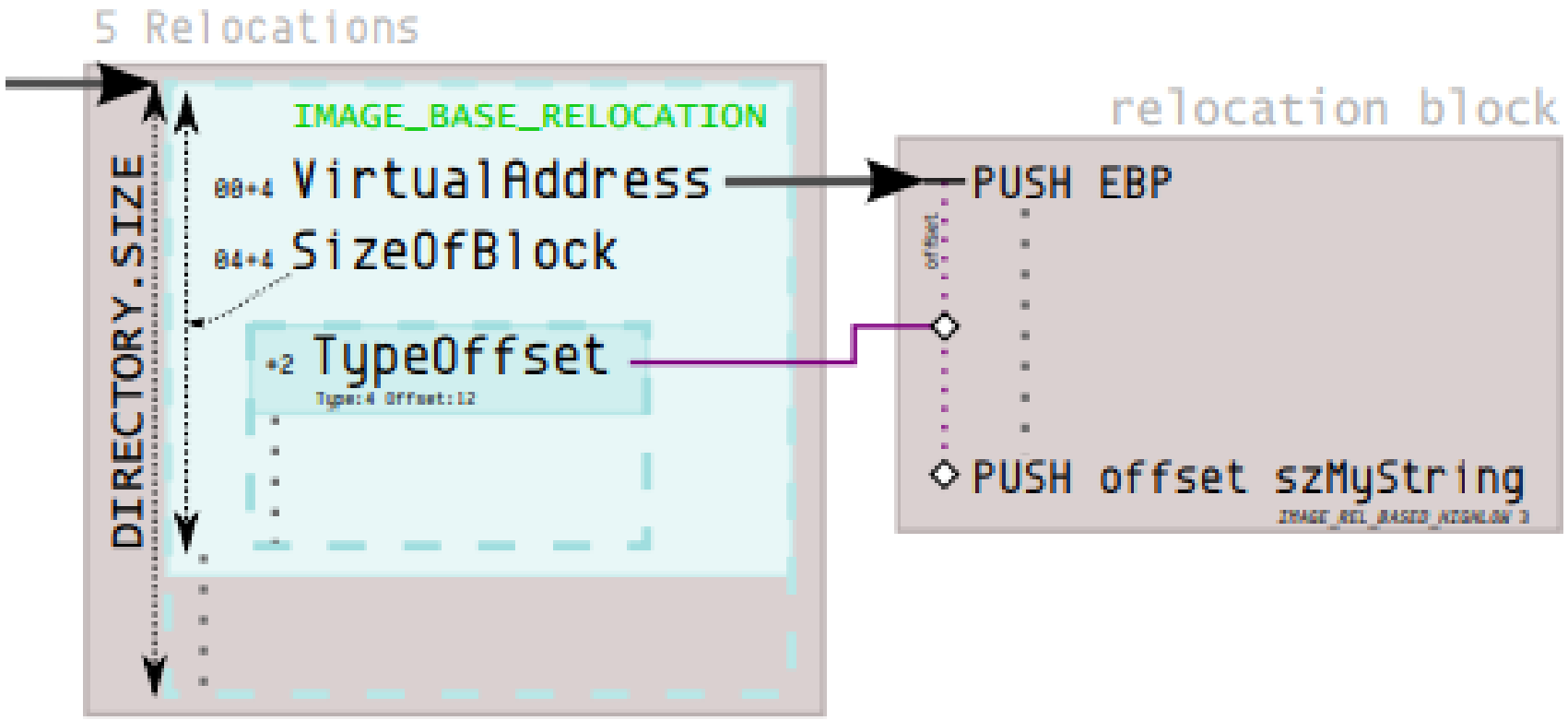

Resources (data itself)
Strings
00+2 length
02+? string
Size
<Resource data>
Version information
VS_VERSION_INFO
00+02 wLength
02+02 wValueLength
04+02 wType
06+2*? szKey[] "VS_VERSION_INFO"
+[0-3] Padding1
??+34 Value
VS_FIXEDFILEINFO
00+4 dwSignature
04+4 dwStrucVersion
08+4 dwFileVersionMS
0c+4 dwFileVersionLS
10+4 dwProductVersionMS
14+4 dwProductVersionLS
18+4 dwFileFlagsMask
1c+4 dwFileFlags
20+4 dwFileOS
24+4 dwFileType
28+4 dwFileSubtype
2c+4 dwFileDateMS
30+4 dwFileDateLS
wValueLength
??+[0-3] Padding2
??+? Children
StringFileInfo
00+2 wLength
02+2 wValueLength
04+2 wType
08+2*? szKey "StringFileInfo"
+[0-3] Padding
??+? Children
StringTable
00+2 wLength
02+2 wValueLength 0 = no value
04+2 wType
08+2*? szKey
+[0-3] Padding
??+? Children
String
00+2 wLength
02+2 wValueLength
04+2 wType
08+2*? szKey
+[0-3] Padding
+2*? Value[]
VarFileInfo
00+2 wLength
02+2 wValueLength 0 = no value
04+2 wType
08+2*? szKey "VarFileInfo"
+[0-3] Padding
??+? Children
Var
00+2 wLength
02+2 wValueLength
04+2 wType
08+2*? szKey "Translation"
+[0-3] Padding
+4*? Value[]
wLength
Icons
<header-less .ICO data>
Group Icons
GRPICONDIR
00+2 idReserved
02+2 idType
04+2 idCount
GRPICONDIRENTRY
00+1 bWidth
01+1 bHeight
02+1 bColorCount
03+1 bReserved
04+2 wPlanes
06+2 wBitCount
08+4 dwBytesInRes
0C+2 nId
Manifest
<XML file>

Exceptions


4 Exceptions
DIRECTORY.SIZE (required)
RUNTIME_FUNCTION
00+4 FunctionStart
04+4 FunctionEnd
08+4 UnwindInfo
UNWIND_INFO
00+1 Version/Flags :3 :5
01+1 SizeOfProlog
02+1 CountOfCodes
03+1 FrameRegister/Offset :4 :4
UNWIND_CODE
00+1 CodeOffset
01+1 UnwindOp/Opinfo :4 :4
02+2 FrameOffset
??+4 ExceptionHandler/FunctionEntry
+4 ExceptionData[]

Security(인증서)

Relocation

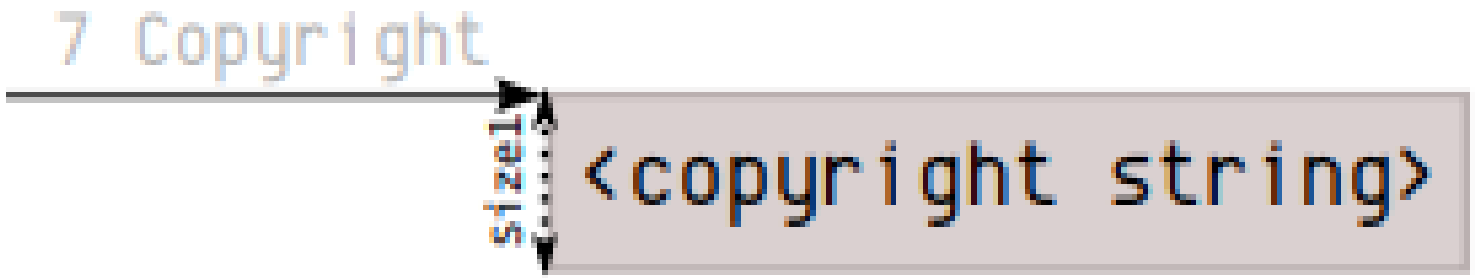

5 Relocations
DIRECTORY.SIZE
IMAGE_BASE_RELOCATION
00+4 VirtualAddress
04+4 SizeOfBlock
+2 TypeOffset
Type:4 Offset:12
relocation block
PUSH EBP
offset
PUSH offset szMyString

Debug

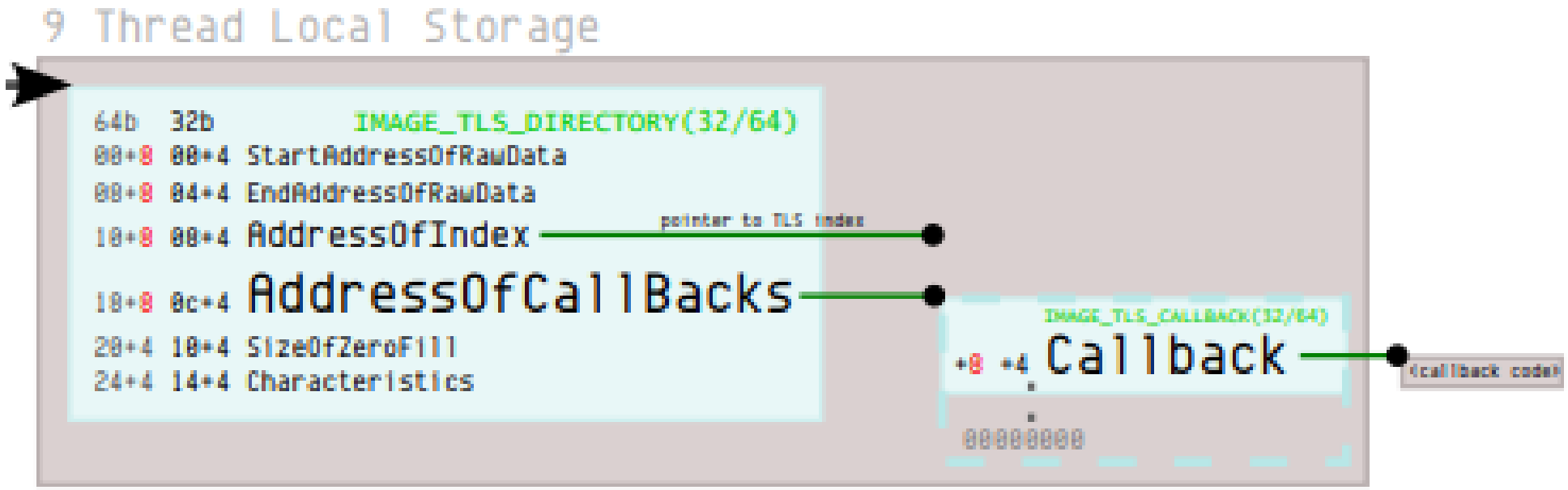
6 Debug symbols

```
      IMAGE_DEBUG_DIRECTORY
00+4 Characteristics
04+4 TimeDateStamp
08+2 MajorVersion
0a+2 MinorVersion
0c+4 Type  1 Coff/2 CV-PDB/9 Borland
10+4 SizeOfData
14+4 AddressOfRawData
18+4 PointerToRawData
```

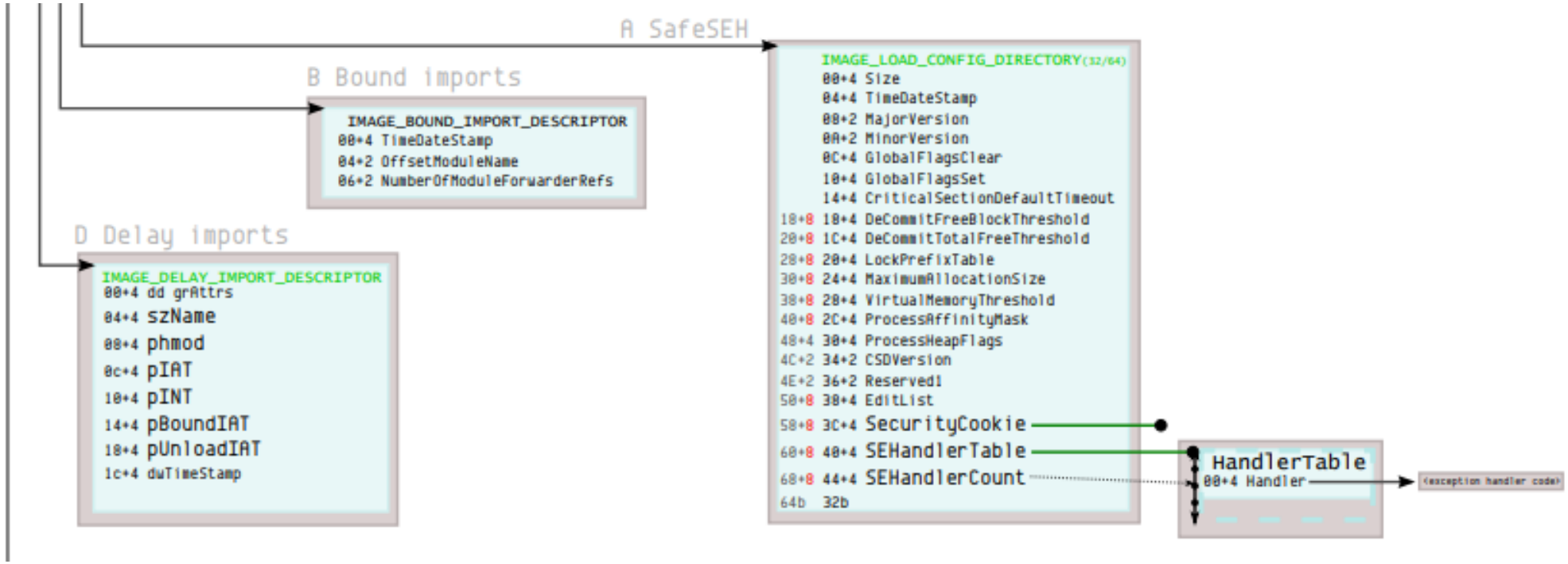
COPYRIGHT/Architecture



TLS

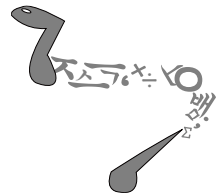

9 Thread Local Storage
64b 32b IMAGE_TLS_DIRECTORY(32/64)
00+8 00+4 StartAddressOfRawData
08+8 04+4 EndAddressOfRawData
10+8 08+4 AddressOfIndex
pointer to TLS index
18+8 0c+4 AddressOfCallBacks
20+4 10+4 SizeOfZeroFill
24+4 14+4 Characteristics
IMAGE_TLS_CALLBACK(32/64)
+8 +4 Callback
00000000
<callback code>

그외


A SafeSEH
B Bound imports
IMAGE_BOUND_IMPORT_DESCRIPTOR
00+4 TimeDateStamp
04+2 OffsetModuleName
06+2 NumberOfModuleForwarderRefs
D Delay imports
IMAGE_DELAY_IMPORT_DESCRIPTOR
00+4 dd grAttrs
04+4 szName
08+4 phmod
0c+4 pIAT
10+4 pINT
14+4 pBoundIAT
18+4 pUnloadIAT
1c+4 dwTimeStamp
IMAGE_LOAD_CONFIG_DIRECTORY(32/64)
00+4 Size
04+4 TimeDateStamp
08+2 MajorVersion
0A+2 MinorVersion
0C+4 GlobalFlagsClear
10+4 GlobalFlagsSet
14+4 CriticalSectionDefaultTimeout
18+8 18+4 DeCommitFreeBlockThreshold
20+8 1C+4 DeCommitTotalFreeThreshold
28+8 20+4 LockPrefixTable
30+8 24+4 MaximumAllocationSize
38+8 28+4 VirtualMemoryThreshold
40+8 2C+4 ProcessAffinityMask
48+4 30+4 ProcessHeapFlags
4C+2 34+2 CSDVersion
4E+2 36+2 Reserved1
50+8 38+4 EditList
58+8 3C+4 SecurityCookie
60+8 40+4 SEHandlerTable
68+8 44+4 SEHandlerCount
64b 32b
HandlerTable
00+4 Handler
<exception handler code>

Chapter. • 5

Q&A



Q&A